

Glance ML Internship Assignment

Multimodal Fashion & Context Retrieval

Glance ML Internship Assignment

Multimodal Fashion & Context Retrieval

Submission date: July 2026

Repository
[github.com]

1. Approaches Considered

This section enumerates the plausible solutions to the problem, with tradeoffs.

1.1 Vanilla CLIP (baseline, rejected)

A single ViT-B-32 encoder embeds both images and natural-language queries. Cosine similarity scores the top-k.

Pros	Cons
Trivial to implement	Explicitly fails compositionality (red tie vs. blue tie)
Small index, fast search	Out-of-distribution on fashion attributes
Many open-source checkpoints	Single-vector similarity collapses fine-grained signal

| Trivial to

Verdict: Discarded. The assignment brief itself warns against this exact approach.

1.2 Fashion-tuned CLIP encoder

Replace ViT-B-32 with a fashion-domain model: Marqo/marqo-fashionCLIP (ViT-L/14, open CLIP-compatible via hf-hub:), or a SigLIP variant trained on fashion data.

Pros	Cons
Trivial to implement	Explicitly fails compositionality (red tie vs. blue tie)
Small index, fast search	Out-of-distribution on fashion attributes
Many open-source checkpoints	Single-vector similarity collapses fine-grained signal

|---|---|

| Trivial to

Verdict: Adopted as the embedding backbone (default ViT-B-16-SigLIP-512, with hf-hub:Marqo/marqo-fashionCLIP flagged as alternative).

1.3 Caption-augmented dual-vector retrieval

For every image, generate a natural-language caption with BLIP-base. Embed captions with the same CLIP text encoder. Compute hybrid score:

```
score(q, img) = ? * cosine(q, image_emb)
               + ? * cosine(q, caption_emb)
```

Pros	Cons
Trivial to implement	Explicitly fails compositionality (red tie vs. blue tie)
Small index, fast search	Out-of-distribution on fashion attributes
Many open-source checkpoints	Single-vector similarity collapses fine-grained signal

|---|---|

| Trivial to

Verdict: Adopted - the core defense against compositionality.

1.4 Late-interaction (ColBERT-style) over caption tokens

Instead of a single caption embedding, embed each token and compute max-similarity over tokens between query and caption.

| Pros | Cons |

```
|---|---|
| Trivial to implement | Explicitly fails compositionality (red tie vs. blue tie) |
| Small index, fast search | Out-of-distribution on fashion attributes |
| Many open-source checkpoints | Single-vector similarity collapses fine-grained signal |

|---|---|
```

| Trivial to

Verdict: Deferred to future work. Marginal gain on a 3.2k corpus not worth the complexity.

1.5 Cross-encoder re-ranking

After retrieving top-N=50 candidates via ?1.3, score each (query, caption|text) pair with a text cross-encoder (ms-marco-MiniLM-L-2-v2). Promote the most relevant.

```
| Pros | Cons |
|---|---|
| Trivial to implement | Explicitly fails compositionality (red tie vs. blue tie) |
| Small index, fast search | Out-of-distribution on fashion attributes |
| Many open-source checkpoints | Single-vector similarity collapses fine-grained signal |

|---|---|
```

| Trivial to

Verdict: Adopted - flag-toggled, defaults to on.

1.6 Selected architecture

```
| # | Component | Choice |
|---|---|---|
| Embedder | Fashion-aware OpenCLIP | `ViT-B-16-SigLIP-512 (webli)` default; `hf-hub:Marqo/marqo-fashionCLIP` opt-in
|
| Image index | FAISS `IndexFlatIP` | exact cosine on normalized vectors |
| Caption generator | `Salesforce/blip-image-captioning-base` | offline, batched, resumable |
| Caption index | FAISS `IndexFlatIP` over caption embeddings | one row per image |
| Hybrid score | Weighted sum (image + caption) | `?`,`?` configurable |
| Re-ranker | `cross-encoder/ms-marco-MiniLM-L-2-v2` | on top-N=50 candidates |
| Scale-up path | `IndexIVFFlat` with PQ | one CLI flag away |
```

|---|---|---|

| Embe
hf-hub:Ma

2. Chosen Architecture - Detail

2.1 Data flow

```

+-----+
| Query decomposition |
| (none; rely on embeddings) |
+-----+
| raw text |
| ? |
+-----+
| OpenCLIP text encoder |
| ViT-B-16-SigLIP-512 (webli) |
+-----+
| 1 x D |
| ? |
+-----+
| FAISS IndexFlatIP (img) | ---> top-N image indices
| FAISS IndexFlatIP (cap) | ---> top-N caption indices
+-----+
| 2N candidates |
| ? |
+-----+
| Hybrid scoring |
| ? * img_sim + ? * cap_sim |
+-----+
| top-N=50 |
| ? |
+-----+
| Cross-encoder re-ranker |
| ms-marco-MiniLM-L-2-v2 |
+-----+
| ranked list |
| ? |
+-----+
| top-k results |
+-----+

```

Offline (one-time) indexer:

```

images/          caption generator (BLIP-base)
|               |
|----? OpenCLIP image -----+   `----? JSON cache (resumable)
|      encoder              |       |
|                          ?       ?
|          faiss.index      faiss.index (caption)
+-----?      metadata.json   caption_meta.json

```

2.2 Why this beats vanilla CLIP on the rubric

```

| Rubric concern | Defense |
|---|---|
| Compositionality ("red tie + white shirt, formal") | Caption augmentation lifts the constraint into natural language; cross-encoder evaluates (query, caption_text) together |
| Fine-grained attributes (silk, denim, navy) | Fashion-aware OpenCLIP backbone (`ViT-B-16-SigLIP`) was trained on a much larger web corpus than the rubric's expected data |
| "Casual weekend for a city walk" -> style inference | Captions contain style vocabulary; cross-encoder maps lifestyle intent to wardrobe semantics |
| Multi-attribute queries (color + clothing + location) | Each axis is tokenized and weighted into a single embedding; cross-encoder disambiguates final ranking |
| Zero-shot generalization | No fine-tuning, no labeled data needed |
|---|---|

```

| Trivial to

2.3 Codebase map

```
src/glance_search/  
  config.py          YAML + env-override config (single source of truth)  
  errors.py          domain exceptions  
  logging_setup.py   logger setup  
  model.py           OpenCLIP wrapper (singleton cache, auto-dim)  
  embedder.py        batched image embedding, L2-normalized, error-tolerant  
  captions.py        BLIP-base offline captioning (resumable)  
  reranker.py        cross-encoder re-ranker  
  index_store.py     FAISS IndexFlatIP + IndexIVFFlat (toggle via --backend)  
  pipeline.py        end-to-end search orchestration (image+captions+reranker)  
  
indexer/build_index.py  CLI: embed images -> faiss.index  
retriever/search.py     CLI: text query -> top-k  
scripts/caption_corpus.py CLI: BLIP captions for all images  
scripts/build_caption_index.py CLI: embed captions -> captions.index  
scripts/run_eval.py     CLI: 5-rubric-query harness -> JSON + PNG grids  
scripts/run_ablation.py CLI: A/B comparison of configs -> ablation.csv  
scripts/build_report.py CLI: markdown writeup -> HTML / PDF  
app/streamlit_app.py    Streamlit interactive demo  
tests/                  pytest suite, no model download needed  
report/                  this file (Glance_Internship_Report.{md,html,pdf})  
eval/results/            generated by run_eval (created at run time)  
output/                  generated indexes + cache
```

3. Evaluation Results

The five rubric queries from ?4 of the assignment brief:

#	Type	Query
Q1	Attribute Specific	"A person in a bright yellow raincoat."
Q2	Contextual / Place	"Professional business attire inside a modern office."
Q3	Complex Semantic	"Someone wearing a blue shirt sitting on a park bench."
Q4	Style Inference	"Casual weekend outfit for a city walk."
Q5	Compositional	"A red tie and a white shirt in a formal setting."

Embed
hf-hub:Ma

For each query, the pipeline returns top-k=5 images. Result grids are persisted to eval/results/<queryslug>.png and the JSON dump to eval/results/<queryslug>.json.

3.1 Result grids

#	Query	Top-5 grid
Q1	A person in a bright yellow raincoat.	![Q1](eval/results/01_yellow_raincoat.png)
Q2	Professional business attire inside a modern office.	![Q2](eval/results/02_business_office.png)
Q3	Someone wearing a blue shirt sitting on a park bench.	![Q3](eval/results/03_blue_shirt_park.png)
Q4	Casual weekend outfit for a city walk.	![Q4](eval/results/04_casual_city.png)
Q5	A red tie and a white shirt in a formal setting.	![Q5](eval/results/05_red_tie_white_shirt.png)

Ablation rows (produced by scripts/runablation.py, persisted to eval/results/ablation.csv):

Config	Q1 yellow raincoat	Q2 business office	Q3 blue shirt park	Q4 casual city	Q5 red tie white shirt	mean top-1	? vs prev
image_only` (vanilla CLIP)	0.088	0.088	0.109	0.102	0.049	0.087	-
image_captions` (+ BLIP captions)	0.335	0.349	0.321	0.305	0.301	0.322	+270 %
image_captions_rerank` (+ cross-encoder)	0.660	0.174	0.622	0.153	0.304	0.382	+19 %

- Q1 yellow raincoat and Q3 blue shirt on a park bench - strong visual concepts, image + caption + rerank all materially help (Q1 jumps 0.088 -> 0.660, a 7.5x lift).

- Q2 bu
alone gives
hybrid stag
recover this

3.1 Failure case discussion

For the imageonly baseline, the most likely failure modes are:

- Q1 ("bright yellow raincoat") - vanilla CLIP associates raincoat with dark/somber tones and surfaces dark coats. With captions, "yellow" is paired with "raincoat" in BLIP descriptions, lifting the score 3.8x.

- Q3 ("blu
retrieval re
composites

The cross-encoder's mixed impact on Q2 and Q4 is the largest single improvement opportunity. The fix is the smallest: increase retrieval.reranktopn from 50 to 100, or re-rank from the top-200 of the *caption* index instead of the hybrid.

4. Future Work

4.1 Locations, cities, weather (per assignment ?5.4a)

The current system treats the world as a flat image-text embedding space. To extend it with location / weather awareness:

1. Caption enrichment

For each image, in addition to garment description, also store:

- scene type (urban / coastal / forest / indoor)
- weather (sunny / rainy / overcast / snowy)
- inferred city (multinomial from a geolocation head)

Use a vision-language model (Florence-2, LLaVA-1.5, CogVLM) on top of the same encoder pipeline.

2. Geo prior

Index city + month -> weather priors (Open-Meteo API).

At query time, intersect query tokens (e.g., "coastal street in Mumbai, August") with the priors and re-weight candidates that match.

3. Retrieval API

/search?q=...&city=mumbai&month=august -> merged vector + geo-temporal filter.

This is also the foundation for "shop the look in my city" applications on Glance's lock-screen.

4.2 Improving precision (per assignment ?5.4b)

A precision improvement roadmap:

Lever	Mechanism	Estimated gain
Larger encoder	`ViT-L-14-SigLIP2-512` over `B-16`	+2 to +5 nDCG@10 on public fashion datasets
Hard-negative mining	Mine compositional swaps within the corpus, contrastive-fine-tune for 1k steps	+5 to +8 on Q5 specifically
Late-interaction (ColBERT-style)	on caption tokens	Token-level max-similarity +3 to +6 on compositional queries
Generative re-ranker (LLM judge)	LLaVA-1.6 evaluates (query, image) pair, output ? {0, 1}	+5 to +10 but higher latency
Active learning feedback loop	Capture click-through on top-k; add as positive pairs in monthly retraining	compounding, +1 to +3 per cycle
Multi-modal product metadata	Add brand/title/price as side-text encodings; hybrid at index time	+1 to +3

| Embe
hf-hub:Ma

Highest-ROI single addition: hard-negative fine-tune. The model already has fashion priors; teaching it to distinguish "red tie + white shirt" from "white tie + red shirt" demands labeled pairs more than architecture.

Appendix A. Codebase

GitHub: [\[github.com/amanraj74/Glance-fashion-search\]](https://github.com/amanraj74/Glance-fashion-search)(<https://github.com/amanraj74/Glance-fashion-search>)

The repository contains the full pipeline - indexer, retriever, scripts, Streamlit app, tests, and this writeup. Local reproduction instructions are in Appendix B below.

Appendix B. Reproducibility

All commands are run from the repository root with the venv activated.

```
.\env\Scripts\Activate.ps1

# 1. Build the image index (downloads ~500 MB ViT-B-16-SigLIP-512 weights)
python indexer/build_index.py

# 2. Generate BLIP captions and embed them (~500 MB BLIP-base weights)
python scripts/build_caption_index.py

# 3. Run the 5-rubric evaluation harness
python scripts/run_eval.py

# 4. Compare configs
python scripts/run_ablation.py

# 5. Build the PDF writeup (needs `pip install weasyprint` for PDF)
pip install weasyprint
python scripts/build_report.py

# 6. Run the interactive demo
pip install streamlit
```

```
streamlit run app/streamlit_app.py
```

Expected wall-clock on a CPU-only machine:

```
| Step | Wall time | Output |
|---|---|---|
| `indexer/build_index.py` | 15 - 25 min | `output/faiss.index` |
| `scripts/build_caption_index.py` | 30 - 60 min | `output/captions.json`, `output/captions.index` |
| `scripts/run_eval.py` | < 1 min | `eval/results/*.json`, `*.png` |
| `scripts/run_ablation.py` | < 1 min | `eval/results/ablation.csv` |
| `scripts/build_report.py` | < 30 s | `report/*.html`, `*.pdf` |

|---|---|---|
```

| Embe
hf-hub:Ma

Appendix C. Limitations & Honest Tradeoffs

- Compute: CPU-only training environment, so no fine-tuning was performed. The system is strong out of the box (fashion-tuned embedding + caption augmentation + cross-encoder re-ranking) but cannot be made stronger through training without GPU access.

- Dataset
IndexIVFFI